Lecture 9: Hadoop Ecosystem Overview (Hive & HBase)

DSL351 / DSP352

Amit Kumar Dhar

ED1, 406A
amitkdhar@iitbhilai

January 27, 2026

The Hadoop Ecosystem

The Big Data Landscape

- **Core Hadoop:** HDFS (Storage) + YARN (Resource Management) + MapReduce (Processing).
- **Ecosystem:** Specialized tools built on top of Core Hadoop to solve specific problems.
- **Today's Focus:**
  1. **Apache Hive:** SQL-like query interface for data warehousing.
  2. **Apache HBase:** Real-time, random access NoSQL database.

Why Ecosystem Tools?

MapReduce Limitations:

- Verbose Java code.
- High barrier to entry for analysts.
- Batch-only (high latency).
- No random read/write support.

Solutions:

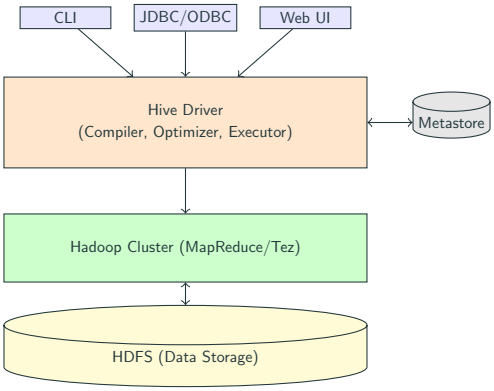
- **Hive:** "I know SQL, I want to query HDFS data."
- **HBase:** "I need to look up a specific user ID in milliseconds."

Apache Hive

What is Apache Hive?

- Data Warehousing software project built on top of Hadoop.
- Provides a SQL-like interface (**HiveQL**) to query data stored in HDFS.
- **Mechanism:** Translates SQL queries into MapReduce (or Tez/Spark) jobs.
- **Metastore:** Stores schema information (table names, columns, partitions) in a relational DB (e.g., MySQL, Derby).

Hive Architecture



Hive Data Model

1. **Tables:** Analogous to RDBMS tables. Maps to a directory in HDFS.
2. **Partitions:** Divides a table into parts based on partition keys (e.g., ‘date‘, ‘country‘). Maps to sub-directories.
3. **Buckets:** Divides partitions into files based on a hash of a column (e.g., ‘userid‘).

File Formats: TextFile, SequenceFile, ORC, Parquet, Avro.

Managed vs. External Tables

Managed (Internal) Tables:

- Hive controls lifecycle of data.
- `DROP TABLE` deletes metadata AND data in HDFS.

```
CREATE TABLE managed_users (id INT, name STRING);
```

External Tables:

- Data exists outside Hive (e.g., raw logs).
- `DROP TABLE` deletes **only** metadata. Data remains safe.

```
CREATE EXTERNAL TABLE ext_logs (line STRING)
LOCATION '/data/logs/webserver';
```

HiveQL: DDL Example

```
CREATE TABLE sales (
    txn_id INT,
    product STRING,
    amount DOUBLE
)
PARTITIONED BY (txn_date STRING)
STORED AS PARQUET;

-- Loading Data
LOAD DATA LOCAL INPATH '/tmp/sales_2023.csv'
INTO TABLE sales PARTITION (txn_date='2023-01-01');
```

Creating a Partitioned Table

HiveQL: Complex Types

Hive supports complex data types: `ARRAY`, `MAP`, `STRUCT`.

```
CREATE TABLE employees (
    name STRING,
    subordinates ARRAY<STRING>,
    deductions MAP<STRING, FLOAT>,
    address STRUCT<street:STRING, city:STRING, zip:INT>
);

-- Querying Complex Types
SELECT name, subordinates[0], deductions['Tax'], address.city
FROM employees;
```

Complex Types Example

HiveQL: Analysis (Word Count)

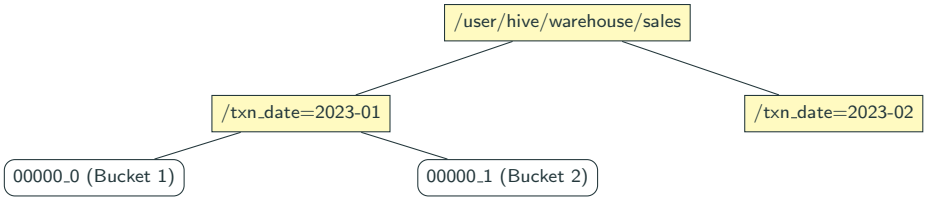
```
-- 1. Create table for raw lines
CREATE TABLE docs (line STRING);

-- 2. Load data
LOAD DATA INPATH '/input/books' INTO TABLE docs;

-- 3. Explode and Count
SELECT word, COUNT(1) as freq
FROM docs
LATERAL VIEW explode(split(line, ' ')) lv AS word
GROUP BY word
ORDER BY freq DESC;
```

Word Count in Hive

Partitions and Buckets Illustration



- **Partitioning:** Prunes directories during query (Filesystem optimization).
- **Bucketing:** Efficient sampling and Joins (Data organization).

Optimizing Hive Queries

1. **Partition Pruning:** Always filter by partition key in `WHERE` clause.
2. **Vectorization:** `set hive.vectorized.execution.enabled = true;` (Process 1024 rows at a time).
3. **Cost Based Optimizer (CBO):** Uses stats to pick join type.
4. **File Format:** Prefer **ORC** or **Parquet** over Text.

Hive User Defined Functions (UDF)

Extend HiveQL with Java.

```java
package com.example.hive;
import org.apache.hadoop.hive.ql.exec.UDF;

public class ToUpper extends UDF {
    public String evaluate(String s) {
        if (s == null) return null;
        return s.toUpperCase();
    }
}
```

Simple UDF Skeleton

Usage in Hive:

```
ADD JAR my-udf.jar;
CREATE TEMPORARY FUNCTION my_upper AS 'com.example.hive.ToUpper';
SELECT my_upper(name) FROM employees;
```

Apache HBase

What is Apache HBase?

- **Definition:** Distributed, scalable, Big Data store.
- **Nature:**
  - **NoSQL:** Not relational, no joins, no complex transactions.
  - **Column-Oriented:** Stores data by columns, not rows (sort of).
  - **Key-Value Store:** Every piece of data is indexed by a Key.
- **Based on:** Google BigTable paper.
- **Use Case:** Random real-time read/write access to Big Data.

HBase vs. RDBMS vs. HDFS

Feature	RDBMS (MySQL)	HDFS	HBase
Data Size	GB to TB	PB	PB
Access	Random	Sequential	Random
Latency	Low	High	Low
Schema	Strict	Schema-on-Read	Flexible
Updates	Yes	No (Append only)	Yes (Versioning)

HBase Data Model

A "Multi-dimensional sorted map".

- **Table:** Collection of rows.
- **RowKey:** Unique identifier (Byte array). Sorted lexicographically.
- **Column Family:** Group of columns stored together physically. Defined at creation.
- **Column Qualifier:** Specific column within a family (Dynamic).
- **Cell:** Intersection of Row, Column Family, and Qualifier. Contains a Value.
- **Timestamp:** Version of the value (Time-to-Live support).

Visualizing HBase Data Model

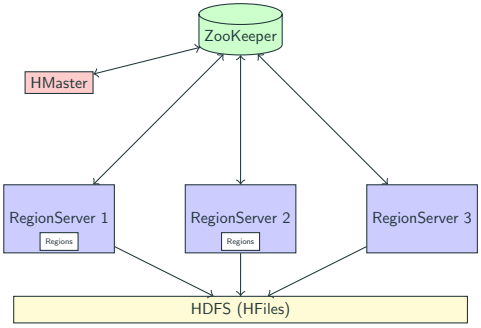
Logical View:

RowKey	CF:Personal	CF:Office
user1	name:Amit	role:Dev, desk:A1
user2	name:John, age:25	role:Mgr

Physical Storage (Key-Value Pairs):

```
user1:Personal:name:t1 -> Amit
user1:Office:role:t1   -> Dev
user1:Office:desk:t1   -> A1
user2:Personal:name:t2 -> John
user2:Personal:age:t2  -> 25
```

HBase Architecture



- **HMaster:** Table metadata, Load balancing (assigning regions).
- **RegionServer:** Serves data requests (Get, Put, Scan).
- **ZooKeeper:** Coordination, Failure detection.

HBase Shell Operations

```
# 1. Create Table (name, column families)
create 'users', 'info', 'logs'

# 2. Insert Data (Put)
put 'users', 'u101', 'info:name', 'Alice'
put 'users', 'u101', 'info:city', 'Delhi'

# 3. Read Data (Get)
get 'users', 'u101'

# 4. Scan Table
scan 'users'

# 5. Disable and Drop
disable 'users'
drop 'users'
```

Basic HBase Shell

HBase Java API: Connecting

```
Configuration config = HBaseConfiguration.create();
config.set("hbase.zookeeper.quorum", "zk1,zk2,zk3");

try (Connection connection = ConnectionFactory.createConnection(config)) {
    Admin admin = connection.getAdmin();

    % Check if table exists
    TableName tableName = TableName.valueOf("users");
    if (!admin.tableExists(tableName)) {
        % Create table logic...
    }
}
```

Connection Setup

HBase Java API: Put Operation

```
Table table = connection.getTable(TableName.valueOf("users"));

% RowKey = "u101"
Put p = new Put(Bytes.toBytes("u101"));

% Family, Qualifier, Value
p.addColumn(
    Bytes.toBytes("info"),
    Bytes.toBytes("name"),
    Bytes.toBytes("Alice")
);

table.put(p);
table.close();
```

Inserting Data

HBase Java API: Scan Operation

```
Table table = connection.getTable(TableName.valueOf("users"));

Scan scan = new Scan();
% Start and Stop Row for optimization
scan.setStartRow(Bytes.toBytes("u100"));
scan.setStopRow(Bytes.toBytes("u200"));

ResultScanner scanner = table.getScanner(scan);
for (Result result : scanner) {
    byte[] val = result.getValue(Bytes.toBytes("info"), Bytes.toBytes("name"));
    System.out.println("Name: " + Bytes.toString(val));
}
scanner.close();
```

Scanning Data

HBase Schema Design: RowKey

The Golden Rule: Access is fast *only* by RowKey.

Strategies:

1. **Salting:** Adding a random prefix to RowKey to prevent "Hotspotting" (all writes going to one server).
2. **Hashing:** Hash the key to distribute evenly.
3. **Reverse Timestamp:** `UserID + (Long.MAX - timestamp)`. Ensures most recent records are fetched first in a scan.

HBase Schema Design: Tall vs. Wide

Tall Table (Preferred):

- Many rows, few columns.
- Example: `Row: U1_T1, Val: Login Row: U1_T2, Val: Logout`

Wide Table:

- Few rows, many columns.
- Example: `Row: U1, Col: T1, Val: Login Row: U1, Col: T2, Val: Logout`

HBase handles billions of rows better than millions of columns.

Integration & Pipeline

Hive on HBase

You can query HBase tables using Hive SQL!

Why?

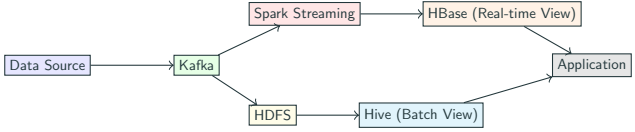
- HBase has no Join support. Hive does.
- Join real-time user data (HBase) with historical logs (Hive/HDFS).

Hive-HBase Mapping Code

```
CREATE EXTERNAL TABLE hive_users(
    key string,
    name string,
    city string
)
STORED BY 'org.apache.hadoop.hive.hbase.HBaseStorageHandler'
WITH SERDEPROPERTIES (
    "hbase.columns.mapping" = ":key,info:name,info:city"
)
TBLPROPERTIES ("hbase.table.name" = "users");
```

Now `SELECT * FROM hive_users` fetches data from HBase.

The Lambda Architecture Pipeline



Summary: When to use what?

- **HDFS:** Cheap storage, high throughput, write-once. (Archival, Data Lake).
- **Hive:** SQL analytics, Batch processing, ETL. (Reporting, Data Science).
- **HBase:** Low latency, Random read/write. (User profiles, Messaging, Real-time dashboards).

Next Class: Introduction to Spark

Transitioning from MapReduce to Memory:

- MapReduce writes to disk after every step. Slow!
- Spark keeps data in RAM. Fast!
- RDDs, DataFrames, and the modern Spark ecosystem.

Questions?